

Operating Systems (OS)

Inter Process Communication (IPC) I

Critical Sections, Synchronisation, Deadlocks

Prof. Dr.-Ing. Jörg Abke



TH Aschaffenburg
university of applied sciences

Copyright Hinweise

© Jörg Abke

Dieses Werk ist urheberrechtlich geschützt. Alle Rechte, insbesondere Vervielfältigung, Verbreitung oder Übersetzung liegen beim Autor. Kein Teil des Werkes darf ohne Genehmigung des Autors, auch nicht für Unterrichtszwecke, fotokopiert oder durch andere Verfahren reproduziert oder verbreitet werden. Ebenfalls ist die Einspeicherung in EDV-Anlagen ohne Genehmigung des Autors nicht gestattet. Jeder Studierende, der an der entsprechenden Lehrveranstaltung des Autors an der Technischen Hochschule Aschaffenburg teilnimmt, ist berechtigt, einen Ausdruck dieses Werkes anzufertigen und für die Zwecke seines Studiums zu verwenden. Eine weitergehende Verwendung bedarf in jedem Falle der ausdrücklichen Zustimmung des Autors. Gebrauchsname, Handelsnamen, Warenbezeichnungen etc. werden in diesem Werk ohne Gewährleistung der freien Verwendbarkeit benutzt. Texte, Abbildungen, Programme und technische Angaben wurden sorgfältig erarbeitet. Trotzdem sind Fehler nicht völlig auszuschließen. Der Autor weist darauf hin, dass er für Fehlerfreiheit keine Gewähr und für eventuelle Folgen keine Haftung übernimmt.

Inter-Process Communication Intro

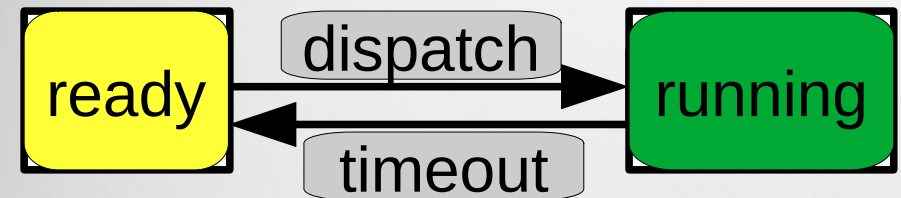
Processes

- Undertake read, write operations on data
- Call each other (e.g. forking)
- Wait for each other

Interaction by processes – Inter-Process Communication (IPC)

→ following questions:

- How to transmit/exchange data
- How to access shared resources

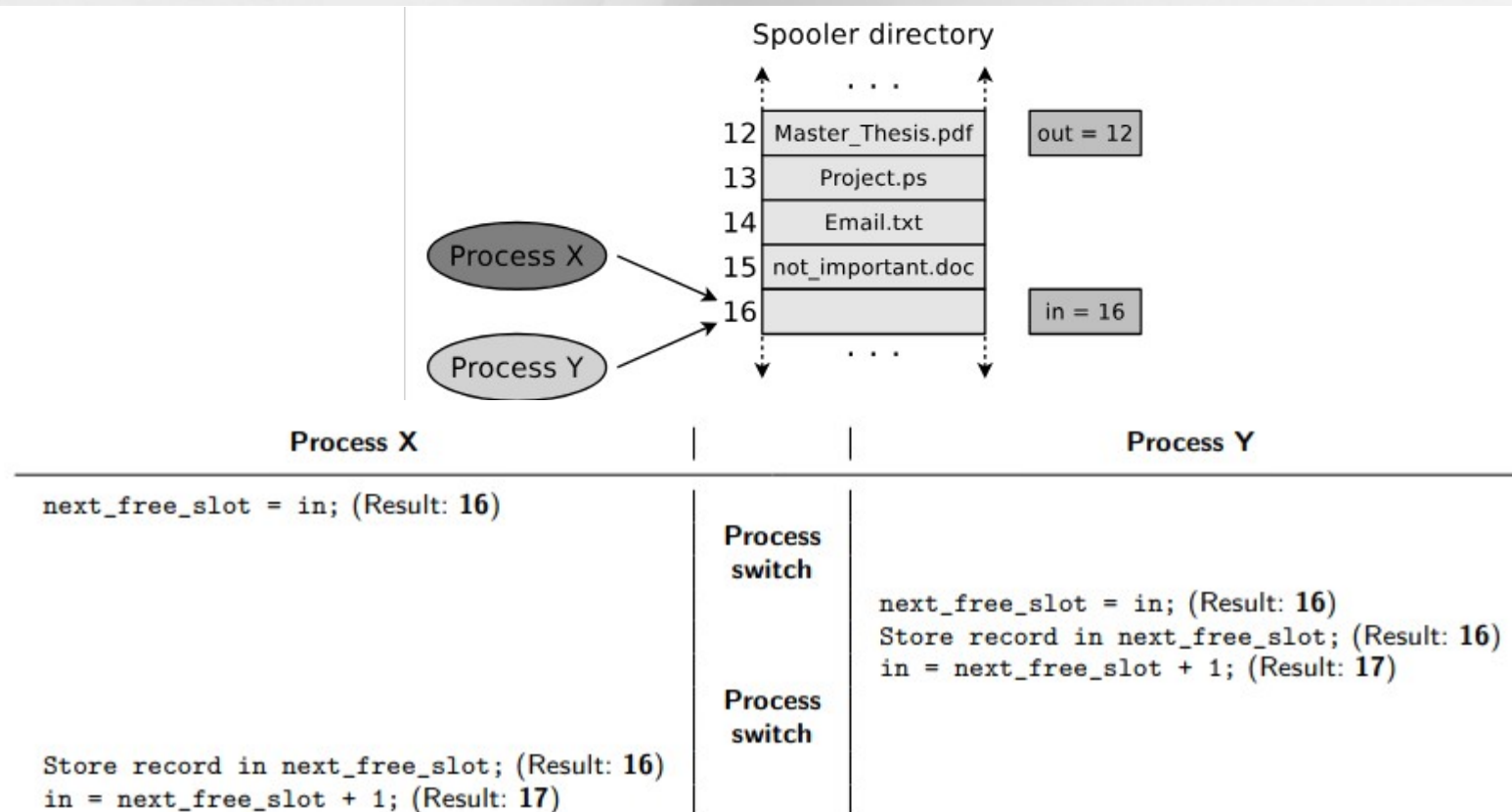


Similar questions for threads

Processes being able to run in parallel (or quasi parallel), they contain

- **Uncritical sections**, where **no access on shared data**
or
only read access on shared data happens
- **Critical sections**, where **read and write operations on shared data**
appear
- **Read & write access to shared memory** by processes necessitates
mutual exclusion, which is provided by Operating System

Critical Section Example: Print Spooler



Source: Chr. Baun, Operating Systems, 3rd ed. Springer-Vieweg, Wiesbaden 2025

Entry (stored record) of process Y is overwritten by stored record of process X
„Race Condition“

Race Condition and a Tragedy with it

- **2 or more processes, which try to modify common data**
 - Result depends on the order or timing of the other processes
 - Correct behaviour of each process, but problem is very hard to locate and find
- **Race conditions** must be **handled by synchronizing** shared data access
- **Tragedy of Therac-25**
Tragic results by race condition in a linear particle accelerator for cancer tumor therapy (between 06/1985 and 01/1987)

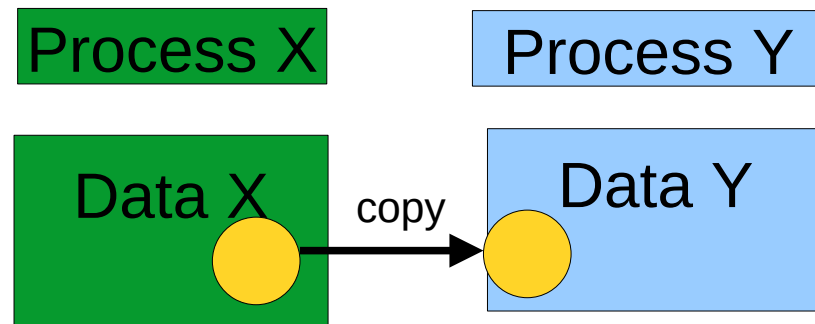
See also: <https://doi.ieeecomputersociety.org/10.1109/MC.2017.4041349>

Communication vs. Cooperation + Synchronization

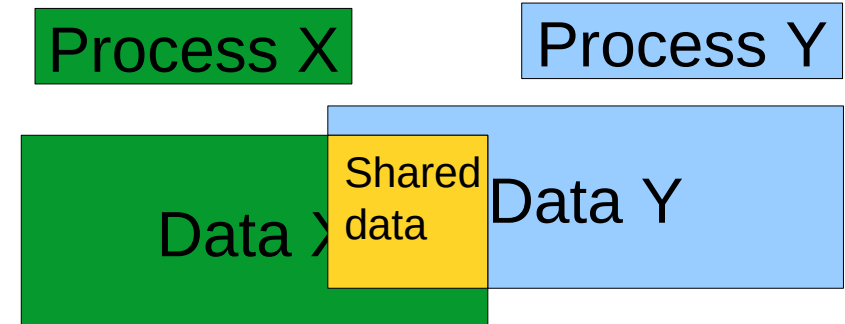


TH Aschaffenburg
university of applied sciences

Communication



Cooperation

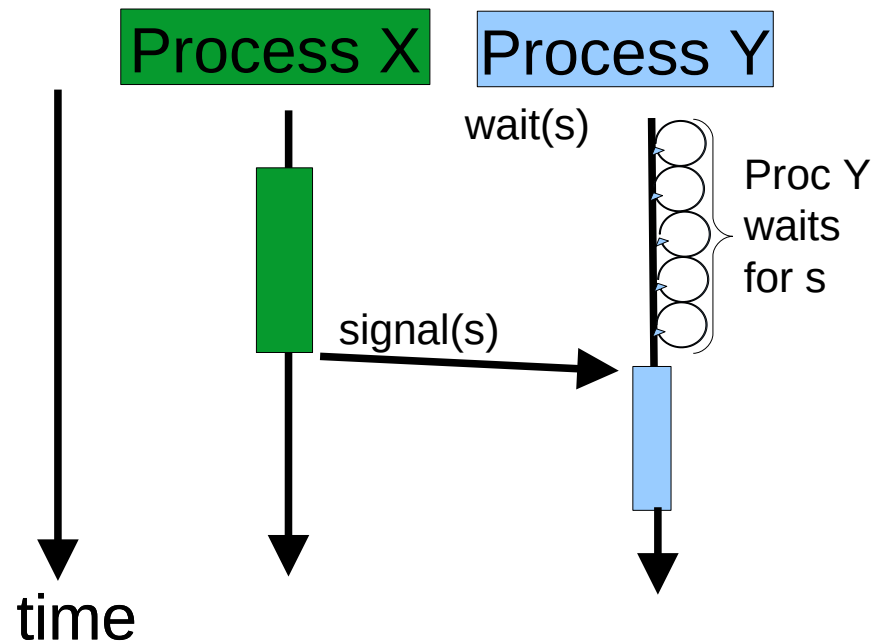


Synchronisation

- **Waiting for other processes**
 - to be finished (→ wait)
 - to give results
- **Execution order of processes**
- **Critical section handling and race condition prevention**

Synchronisation by Signaling – Busy Waiting

Execution order is specified



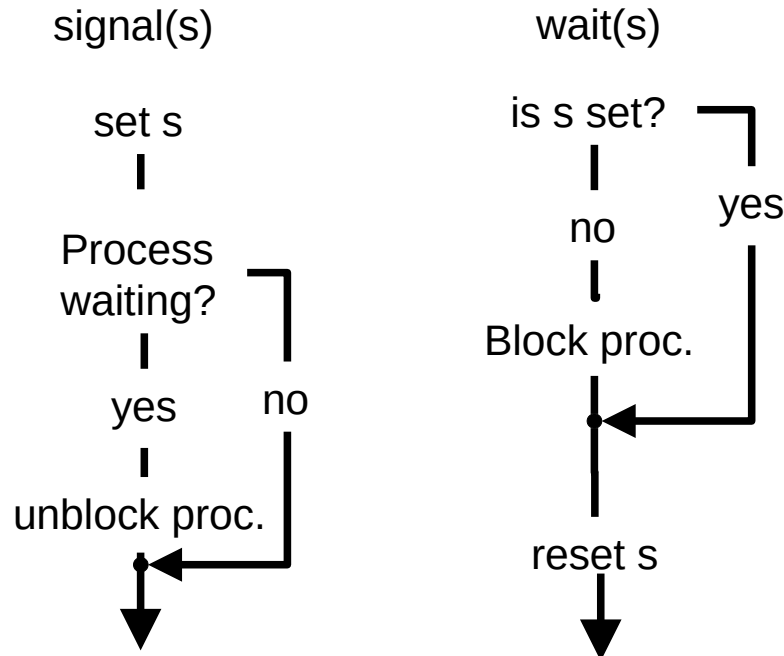
Simple form: Y is **Busy waiting** / **actively waiting** for s

- s might be located in a local file
- Drawback: CPU time is needed for regular checks if s is signaled
- Other term: **polling**

Synchronisation by Signaling – Signal and Wait



TH Aschaffenburg
university of applied sciences



Advanced form: passive waiting, releasing running state and awaiting „wake up signal“

- **Execution order is synchronised**
- Y is blocked until X has finished respective section
- Advantage: No CPU resource is used
- Disadvantage: Only one single process can wait

Example for Process Waiting and Signaling

9_1_process_signaling_for_synchronisation.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
// Signal handler for the child process
void handle_sigusr1(int sig)
{
    printf("Child (%d) received signal %d (SIGUSR1)\n", getpid(), sig);
}
void handle_sigusr2(int sig)
{
    printf("Child (%d) received signal %d (SIGUSR2)\n", getpid(), sig);
}
int main() {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
    if (pid == 0) {
        // --- CHILD PROCESS ---
        printf("Child process started with PID %d\n", getpid());
        // Install signal handler
        signal(SIGUSR1, handle_sigusr1);
        signal(SIGUSR2, handle_sigusr2);
        pause(); // sleeps until a signal arrives
        printf("Child process goes on");
    }
```

```
} else {
    // --- PARENT PROCESS ---
    printf("Parent process (PID %d) sending signal
        to child PID %d...\n",
        getpid(), pid);

    sleep(60); // Give child time to set up
                handler

    // Send SIGUSR1 to child
    kill(pid, SIGUSR1);

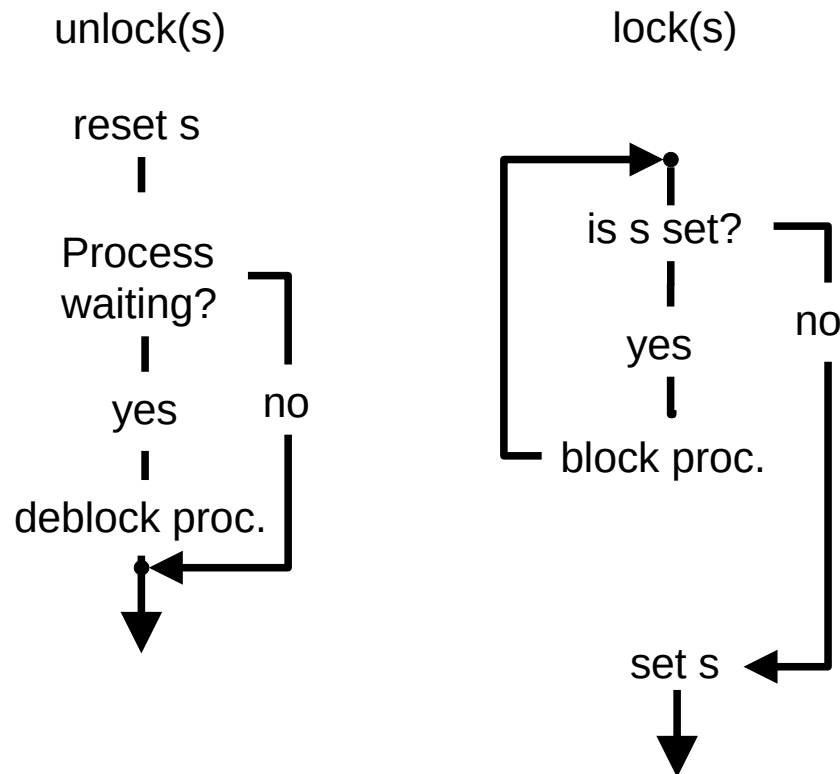
    printf("Parent sent SIGUSR1 to child.\n");

    // Wait for the child (optional)
    wait(NULL);
}

return 0;
}
```

sigsuspend() often more appropriate than pause()
left out due to limited time in this course

lock and unlock

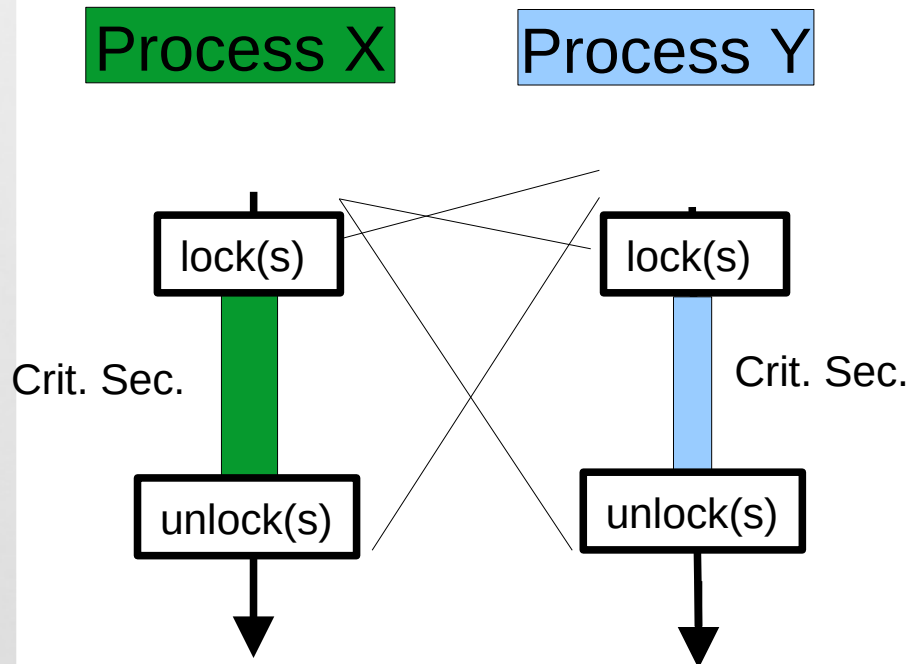


- Preventing from **overlapping execution** of critical sections of two processes
- Unix-like OS system calls
 - sigsuspend, kill, pause and sleep

Protection of Critical Sections with lock and unlock



TH Aschaffenburg
university of applied sciences



- No definition of execution order
- Preventing from **overlapping execution** of critical sections of two processes

- One critical section of X or Y can be run. the are mutually exclusive.

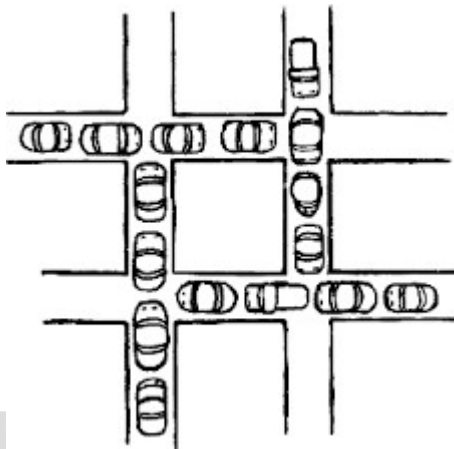
Problems Induced by Blocking

- **Starvation**

- If a process does not remove a lock, all other processes must wait infinitely long time for release
- Similar problem with higher priority processes in scheduling

- **Deadlock**

- If two or more processes wait for the same resource, locked by each other, they lock each other mutually



Source: <https://pradeesh-kumar.medium.com/deadlock-livelock-race-condition-and-starvation-c225018bbae6>

A deadlock occurs if all conditions are fulfilled

- Mutual exclusion
At least 1 resource is occupied by exactly process or is available. The resource can not be shared
- Hold and wait
A process occupies at least 1 resource requests additional resources, which are already occupied by another process
- No Preemption
Resources, which are occupied by a process cannot be deallocated by the OS, but only released by the holding process voluntarily
- Circular wait
 - A cyclic chain of processes exists
 - Each process requests a resource that the next process in the chain occupies

Deadlock Prevention

- **Methods exist for preventing from deadlocks:**
 - Resource Graphs
 - Petri Nets
 - Deadlock detection by matrices
- **All beyond the scope of this lecture**